

```
In [1]: # Initialize Otter
import otter
grader = otter.Notebook("cs1090a_hw0.ipynb")
```



HARVARD
John A. Paulson
School of Engineering
and Applied Sciences

CS1090A Introduction to Data Science

Homework 0

Harvard University

Fall 2026

Instructors: Pavlos Protopapas & Kevin Rader

What HW0 is for

CS1090A attracts students with very different backgrounds — some with years of programming behind them, some fluent in statistics and newer to Python. Homework 0 is how you find out where you stand against the prerequisites, while there is still time to act on it.

You are not expected to recall all of it from memory; looking things up is the job. But if large parts feel not just rusty but genuinely unfamiliar, consider spending more time with the prerequisite material and taking CS1090A in a later semester.

How this is graded

On completion. Attempt every question in a block and the block is yours. Mistakes are fine. Leaving sections blank is not.

Block	Where it is graded	Points
Part 2 — Python Poetry Challenge	Homework 0 — Notebook 📄 (autograded + TF)	35
Part 3 — Math and Statistics	Homework 0 — Notebook 📄 (autograded + TF)	35
Your written work and the Reflection	Homework 0 — PDF 📄 (read by a TF)	30

Part 1 is not graded directly. It is what makes the rest of the notebook run.

The correctness tests are worth zero points. They run and you see every result, but they cannot move your score. What you get back instead is a **diagnostic**: every

question marked *solid*, *shaky* or *not attempted*. That report is the point of the assignment — and it is built from what you wrote, so an answer you didn't understand produces a report about somebody who isn't you.

Work individually, unlike HW1–HW5. HW0 is 1% of the course grade, individual, and **late days do not apply**. You may resubmit either file on Gradescope as often as you like before the deadline.

What to submit

Two files, and you have to submit both. They are the same document in two formats, but they are two separate Gradescope assignments, graded separately and due at the same time.

Upload	What it is	Worth	Graded by
Homework 0 — Notebook 	this notebook, <code>cs1090a_hw0.ipynb</code>	70	an autograder, within a couple of minutes
Homework 0 — PDF 	a PDF of the same notebook	30	a TF, who reads the whole thing

On the PDF, Gradescope asks you to mark which pages six sections of the assignment are on. The closing cell of this notebook shows you exactly what to mark.

Your notebook has to run, from the top to the bottom, and be submitted with its output. Before you export or upload anything, run `Kernel → Restart Kernel` and `Run All Cells`, then save. A notebook that was never run, whose cells were run out of order, or that **stops on a cell that raises** — `Run All` halts there and everything below stays blank — **scores zero on the notebook assignment**, however much is written in it.

Running is not the same as being right. A wrong answer costs you nothing; a cell that raises an error that stops execution costs you the rest of the notebook. If you cannot fix one, put a plausible value in it and say in a comment what you were attempting — then the cells below it have something to work with and your notebook reaches the end. Resubmission is unlimited, so this costs nothing but the upload.

`SETUP.md`, next to this notebook, has the details: how to run it, how to export the PDF, and how the page marking works.

Part 1: Setting the Stage

Your first task is the one this course never stops needing: **a working Python environment on a machine you control**. You will be given environments you can reach from a browser, but everyone is expected to be able to build a comparable one themselves.

This part asks you no questions. The check is simply whether the cell at the end of it runs — if your environment isn't built, nothing else in this notebook will work either.

You need a terminal, and enough comfort with the [filesystem](#) to move around and read and write files.

Why Python?

Python has a clean, readable syntax — "pseudocode you can run" — and a huge scientific ecosystem on top of a large [standard library](#). That combination is why it dominates data science.

The shell and the terminal

Most operating systems already ship some Python. You can ask your [shell](#) — the program that gives you access to the operating system's services — through a [terminal](#):

```
which python
python --version
```

`which` reports the path of the executable that would run if you typed that name, by searching the `PATH` environment variable.

! On some systems you need `python3` rather than `python`. 💡
Prefixing a command with `!` runs it in the shell from inside a notebook.

Windows users: [PowerShell](#) works for most of this course but lacks many standard UNIX commands. Consider [Git Bash](#), or better, [Windows Subsystem for Linux](#) (WSL).

```
In [2]: !python --version
```

```
Python 3.12.13
```

You want 3.10 or newer; the course environment uses 3.12. If `which` found nothing, or the version is older, [install Python](#) first.

Virtual environments

Beyond the standard library we want third-party packages, and *how* you install them matters:

1. Different projects need conflicting versions of the same package, and those versions cannot coexist in one installation.
2. Your operating system may itself depend on Python; changing the system installation can break things you did not intend to touch.

3. Results are not reproducible unless the exact package versions that produced them are recorded.

A [virtual environment](#) solves all three: a self-contained directory holding an interpreter plus its own packages.

This course recommends [uv](#). Install it, then, **from the folder holding this notebook's `requirements.txt`** :

```
uv venv
uv pip install -r requirements.txt
source .venv/bin/activate      # Windows: .venv\Scripts\activate
```

Those first two commands are run once per environment. `deactivate` reverses the third.

Jupyter

Most course material is delivered as [Jupyter notebooks](#) — runnable code embedded in formatted prose. The `requirements.txt` you just installed includes JupyterLab, so with the environment activated:

```
jupyter lab
```

or, without activating it first, `uv run jupyter lab` .

Check your environment

Run the cell below. It imports the packages this course needs and reports the Python that is actually running this notebook — which is not always the one `which python` finds, since the kernel has its own interpreter.

If it raises, your environment is not ready. Go back and build it before continuing; everything from Part 2 onward depends on it.

```
In [3]: # Provided – just run it.
import sys
from importlib.metadata import version

import numpy as np
from scipy import stats
import pandas as pd
import matplotlib.pyplot as plt

print(f"Python {sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}")
print(f"Running {sys.executable}")
for _pkg in ("numpy", "scipy", "matplotlib", "pandas"):
    print(f"  {_pkg:<12} {version(_pkg)}")
```

Python 3.12.13

running /Users/juntangwang/Library/CloudStorage/OneDrive-HarvardUniversity/Documents/2026FA/AC-209A/.venv/bin/python

```
numpy      2.5.3
scipy      1.18.1
matplotlib 3.11.1
pandas     2.3.3
```

`pandas` is course content rather than a prerequisite — you are not expected to know it yet. It is checked here only so you find out now if your environment is incomplete.

Part 2: Python Poetry Challenge

CS1090A assumes you already write Python. To find out whether you do, you'll build a **random poem generator** from a collection of [Emily Dickinson](#) poems published by [Project Gutenberg](#).

Skills exercised: file I/O, strings, slicing, lists, dictionaries, logical operators, iteration, functions, classes, reading documentation, and debugging.

This part is a chain — each question consumes the previous one's output, so an error early on could propagate through the rest of Part 2. For this reason, every question is followed by a **checkpoint** cell that reports how your answer compares to ours and then hands you a known-good value to carry forward.

The checkpoints come from `hw0_utils.py`, sitting next to this notebook. It contains reference implementations of everything you're about to write. You can open it. But bear in mind what you'd be trading: a copied solution (besides being easy to detect) defeats the purpose of HW0, which is to help you assess your preparedness for the course. Use `compare()` to find your bug — that is what it's for.

In [4]: `import hw0_utils`

Q2.1 — Read in the file

The document is a digital transcription of [three early Dickinson collections](#) whose U.S. copyright has expired. It ships with this assignment as plain text at `data/pg12242.txt`. (The opaque filename is a condition of the [Project Gutenberg License](#) included in the file.)

Read the whole file into a string called `text`. Then slice out the first 10,000 characters as `head` and the last 25,000 as `tail`.

Keep your output short. 📖 🧑

`text` is nearly 200,000 characters, and one of your two submissions is a **PDF a TF has to read**. Print slices, never whole documents. Widen a slice while you are exploring if you need to, then narrow it again

before you export. If a cell genuinely needs to show more than ~15 lines, right-click it and choose **Enable Scrolling for Outputs**.

```
In [5]: DATA_PATH = "data/pg12242.txt"

with open(DATA_PATH, encoding="utf-8") as f:
    text = f.read()

head = text[:10000]
tail = text[-25000:]
```

```
In [6]: # The start of the document: Project Gutenberg's header, then the first p
# Widen this slice while you explore; put it back before you export the P
print(head[:300])
```

Project Gutenberg's Poems: Three Series, Complete, by Emily Dickinson

This eBook is for the use of anyone anywhere at no cost and with almost no restrictions whatsoever. You may copy it, give it away or re-use it under the terms of the Project Gutenberg License included with this eBook or online

```
In [7]: # The end of the document: the last poems, then the license
print(tail[-800:])
```

download by the etext year. For example:

<https://www.gutenberg.org/etext06>

(Or /etext 05, 04, 03, 02, 01, 00, 99,
98, 97, 96, 95, 94, 93, 92, 92, 91 or 90)

EBooks posted since November 2003, with etext numbers OVER #10000, are filed in a different way. The year of a release date is no longer part of the directory path. The path is based on the etext number (which is identical to the filename). The path to the file is made up of single digits corresponding to all but the last digit in the filename. For example an eBook of filename 10234 would be found at:

<https://www.gutenberg.org/1/0/2/3/10234>

or filename 24689 would be found at:

<https://www.gutenberg.org/2/4/6/8/24689>

An alternative method of locating eBooks:

<https://www.gutenberg.org/GUTINDEX.ALL>

```
In [8]: grader.check("q2.1")
```

```
Out [8]: q2.1 passed! 🌈
```

Q2.2 — Find where each series begins and ends

Reading the `head` and `tail` you just printed, the document has this structure:

- Project Gutenberg: header and metadata
- Series 1: preface
- **Series 1: poems**
- Series 2: preface
- **Series 2: poems**
- Series 3: preface
- **Series 3: poems**
- Project Gutenberg: footer and license

We want only the bold parts. Recognizing and exploiting structure like this is a large part of data science, and the structure here is marked by three strings that occur nowhere else in a misleading position:

```
In [9]: PREFACE_MARKER = "POEMS\n"           # every preface opens with this line
        FIRST_POEM_MARKER = "I. "           # the first numbered poem after a pre
        LICENSE_MARKER = "End of Project Gutenberg" # the license block at the
```

Write `series_bounds(s, search_from=0)`, which returns the `(start, end)` character indices of the next series' **poem block** at or after `search_from`:

- `start` is the index of the first numbered poem (`FIRST_POEM_MARKER`) that follows the next preface (`PREFACE_MARKER`).
- `end` is the index of the preface after that — **or**, when there is no further preface, the index of the license.

That second case is the only real wrinkle: the third series is followed by the license, not by another preface.

Hints:

- `str.index(sub, start)` returns the index of the first occurrence of `sub` at or after `start`, and raises `ValueError` if there is none.
- `str.find(sub, start)` does the same but returns `-1` instead of raising — handy when "there is none" is a case you want to handle rather than an error.

The docstring gives you the answer for the first series, so you can check yourself immediately.

```
In [10]: def series_bounds(s: str, search_from: int = 0) -> tuple[int, int]:
        ...
        Returns: (start, end) indices of the next series' poem block at or af
        Ex:
        ...     series_bounds(text) -> (6714, 49215)
        ...
        next_preface = s.find(PREFACE_MARKER, search_from)
        if next_preface == -1:
            return None, None
        start = s.find(FIRST_POEM_MARKER, next_preface)
        end = s.find(PREFACE_MARKER, start)
```

```

    if end == -1:
        end = s.find(LICENSE_MARKER, start)
    return start, end

```

```

In [11]: # Apply your function three times, walking forward through the document.
series = []
position = 0
for _ in range(3):
    start, end = series_bounds(text, position)
    series.append(text[start:end].strip())
    position = end

for i, s in enumerate(series, start=1):
    print(f"series {i}: {len(s):,} characters, starts {s[:40]!r}")

```

series 1: 42,494 characters, starts 'I. LIFE.\n\n\nI.\n\nSUCCESS.\n\n[Publ
ished in ''
series 2: 67,086 characters, starts "I. LIFE.\n\n\nI.\n\nI'm nobody! Who
are you?"
series 3: 52,493 characters, starts "I. LIFE.\n\n\nI.\n\nREAL RICHES.\n
\n'T is littl"

```

In [12]: grader.check("q2.2")

```

Out[12]: **q2.2 passed!** 🎉

Checkpoint. The three series get joined into one string, and the single bracketed editor's note in the first poem — the only one in the document — is removed along with the blank line beneath it. That cleanup is done for you.

```

In [13]: poems_text = hw0_utils.canonical_poems_text(text)

print(hw0_utils.compare("series", tuple(series), text))
print(f"\npoems_text: {len(poems_text):,} characters\n")
print(poems_text[:300], "...")

```

series: matches. (3 items)

poems_text: 161,980 characters

I. LIFE.

I.

SUCCESS.

Success is counted sweetest
By those who ne'er succeed.
To comprehend a nectar
Requires sorest need.

Not one of all the purple host
Who took the flag to-day
Can tell the definition,
So clear, of victory,

As he, defeated, dying,
On whose forbidden ear
The distant strains of ...

Q2.3 — Remove the section titles and poem numbers

More of what remains isn't Dickinson's: the editors divided each series into sections (I. LIFE. , II. LOVE. , ...) and numbered the poems within them (I. , II. , III. , ...). Every one of those lines begins with a [roman numeral](#).

First, write `startswith_roman(line)`, returning `True` when `line`'s first whitespace-delimited token is a roman numeral. A token qualifies when all three hold:

1. it ends with a `.`
2. the part before that `.` is **not empty**
3. every character of that part is one of `I`, `V`, `X`, `L`, `M`

`C` (100) and `D` (500) are deliberately excluded. Including them would classify `DID.` and `COME.` as numerals and delete real poem lines; the largest numeral here is nowhere near 100. Condition 2 matters too — a line that begins with a bare `.` is not a numbered section, and `all()` of an empty sequence is `True`, so you have to rule it out on purpose.

The function does not need to reject *invalid* numerals such as `IIIV.`; no such string occurs.

Then use it to build `poems_text_nonum`: `poems_text` with every such line removed, and any leading or trailing whitespace stripped from the result.

```
In [14]: def startswith_roman(line: str) -> bool:
          ...
          Returns: True if `line` starts with a roman numeral and False otherwise
          Ex:
```

```

    startswith_roman("III. NATURE.")          -> True
    startswith_roman("I'm nobody! Who are you?") -> False
    ...
    roman_set = ["I", "V", "X", "L", "M"]
    first_token = line.split(" ")[0]
    if first_token.endswith("."):
        if len(first_token[:-1]) > 0:
            if all(c in roman_set for c in first_token[:-1]):
                return True
    return False

```

```

In [15]: poems_text_nonum = '\n'.join([line for line in poems_text.split("\n") if
print(poems_text_nonum[:400])

```

SUCCESS.

Success is counted sweetest
 By those who ne'er succeed.
 To comprehend a nectar
 Requires sorest need.

Not one of all the purple host
 Who took the flag to-day
 Can tell the definition,
 So clear, of victory,

As he, defeated, dying,
 On whose forbidden ear
 The distant strains of triumph
 Break, agonized and clear!

Our share of night to bear,
 Our share of morning,
 Our blank in bliss to f

```

In [16]: grader.check("q2.3")

```

```

Out[16]: q2.3 passed! 🌟

```

Checkpoint.

```

In [17]: poems_text_nonum_ref = hw0_utils.canonical_poems_text_nonum(text)
print(hw0_utils.compare("poems_text_nonum", poems_text_nonum, text))
poems_text_nonum = poems_text_nonum_ref

```

poems_text_nonum: matches. (159,426 characters)

Q2.4 — Create a list of poems

Now split the text into individual poems. To see the structure you need to see the whitespace, so print the raw string rather than the rendered one:

```
In [18]: from pprint import pprint

pprint(poems_text_nonum[:100])

('SUCCESS.\n'
 '\n'
 'Success is counted sweetest\n'
 'By those who ne'er succeed.\n'
 'To comprehend a nectar\n'
 'Requires so')
```

Consecutive poems are separated by runs of newlines, and exploring the text shows the minimum run between two poems is **six**. Use that to build `poem_list`: the poems in the order they appear, each stripped of leading and trailing whitespace.

Hint: splitting will produce some empty strings. Leave them out.

```
In [19]: poem_list = [p.strip() for p in poems_text_nonum.split("\n"*6) if p.strip()

print(f"There are a total of {len(poem_list)} poems.\n")
print(poem_list[109])
```

There are a total of 445 poems.

LOST.

I lost a world the other day.
Has anybody found?
You'll know it by the row of stars
Around its forehead bound.

A rich man might not notice it;
Yet to my frugal eye
Of more esteem than ducats.
Oh, find it, sir, for me!

```
In [20]: grader.check("q2.4")
```

Out[20]: **q2.4** passed! 🍀

Checkpoint.

```
In [21]: poem_list_ref = hw0_utils.canonical_poem_list(text)
print(hw0_utils.compare("poem_list", poem_list, text))
poem_list = poem_list_ref
```

poem_list: matches. (445 items)

Q2.5 — Create a poem dictionary

Reaching a poem by its position in a list means remembering an arbitrary number. A dictionary is friendlier — but we have to decide what to key it on, and that turns out to be a question about the data rather than about Python.

Roughly half these poems appear under a title, and none of those titles are Dickinson's. She published almost nothing in her lifetime and titled almost none of

it; the titles here were supplied by the editors who prepared these volumes after her death. Dickinson is conventionally cited by **first line** instead, and that is what we will key on — the editors' titles come off.

Build `poems_by_first_line`, mapping **each poem's opening line of verse → the poem**:

- Where the editors supplied a title, drop it (and the blank line under it). The key is the first line of what remains.
- Where they did not, the poem already opens with its verse. The key is its first line.

Either way the key is the first line of the body, so every body starts with its own key.

One helper is provided, since telling an editor's title from a line of verse is fiddly and isn't the point of the question:

```
In [22]: from hw0_utils import has_editor_title, first_line

print(has_editor_title(poem_list[0]), "|", first_line(poem_list[0]))
print(has_editor_title(poem_list[1]), "|", first_line(poem_list[1]))
```

True | SUCCESS.

False | Our share of night to bear,

Note: you need no de-duplication here. The editors' titles collided nine times, but all 445 opening lines are distinct — which is a point in favour of how Dickinson is actually cited.

```
In [23]: poems_by_first_line = {}

for poem in poem_list:
    body = poem[len(first_line(poem)):].rstrip("\n") if has_editor_title(
        poems_by_first_line[first_line(body)] = body

print(poems_by_first_line["Hope is the thing with feathers"])
```

Hope is the thing with feathers
That perches in the soul,
And sings the tune without the words,
And never stops at all,

And sweetest in the gale is heard;
And sore must be the storm
That could abash the little bird
That kept so many warm.

I 've heard it in the chillest land,
And on the strangest sea;
Yet, never, in extremity,
It asked a crumb of me.

```
In [24]: grader.check("q2.5")
```

Out [24]: **q2.5 passed!** 🙌

Checkpoint.

```
In [25]: poems_by_first_line_ref = hw0_utils.canonical_poem_dict(text)
print(hw0_utils.compare("poems_by_first_line", poems_by_first_line, text)
poems_by_first_line = poems_by_first_line_ref
```

poems_by_first_line: matches. (445 entries)

Q2.6 — The long words Dickinson reached for

With the poems in a structure we can ask statistical questions of them — starting with which words recur.

Splitting prose into words is its own small swamp (hyphens, apostrophes, em-dashes, trailing punctuation), so the tokenizer is provided:

```
In [26]: from hw0_utils import tokenize

tokenize("Hope is the thing with feathers--\nThat perches in the soul,")
```

```
Out [26]: ['hope',
           'is',
           'the',
           'thing',
           'with',
           'feathers',
           'that',
           'perches',
           'in',
           'the',
           'soul']
```

Using `tokenize` on the **poem bodies** (the dictionary's values, so editor-supplied titles are excluded), produce two things:

1. `word_counts` — a dict mapping every word to the number of times it occurs.
2. `top_words` — a list of the words that are **at least 11 characters long** and occur **at least 4 times**, sorted by count descending, ties broken alphabetically ascending.

The threshold, rather than a "top 10", is what makes this question have one answer: ten different words occur exactly three times, so any top-10 cut would have to pick four of them arbitrarily.

```
In [27]: from collections import Counter

word_counts = Counter()
for body in poems_by_first_line.values():
    word_counts.update(tokenize(body))

top_words = sorted(
    [w for w, c in word_counts.items() if len(w) >= 11 and c >= 4],
```

```

    key=lambda w: (-word_counts[w], w)
)
word_counts = dict(word_counts)

[(w, word_counts[w]) for w in top_words]

```

```

Out[27]: [('everlasting', 10),
          ('immortality', 10),
          ('butterflies', 9),
          ('superfluous', 5),
          ('countenance', 4),
          ('revelations', 4)]

```

```

In [28]: grader.check("q2.6")

```

```

Out[28]: q2.6 passed! 🚀

```

Q2.7 — Random poem generator

Time to put it together. Complete `PoetryCollection` below by writing its `__init__` method. It takes the author's full name and a poem dictionary, and sets the three attributes described in the docstring. `random_poem` is written for you.

Then instantiate the class with Emily Dickinson's poems and call it `poems`.

```

In [29]: class PoetryCollection:
    """
    Attributes
    -----
    author : str
        full name of author
    collection : dict
        dictionary of (title, poem) key-value pairs
    size : int
        number of poems in collection

    Methods
    -----
    random_poem(seed: int = None) -> str
        returns a random poem; pass a seed for reproducibility (default s
    """

    def __init__(self, author: str, data: dict):
        self.author = author
        self.collection = data
        self.size = len(data)

    def random_poem(self, seed: int = None) -> str:
        rng = np.random.default_rng(seed)
        return str(rng.choice(list(self.collection.values())))

```

```

In [30]: poems = PoetryCollection("Emily Dickinson", poems_by_first_line)

print("Author:", poems.author)
print("Number of poems in collection:", poems.size)
print()
print(poems.random_poem(seed=109))

```

Author: Emily Dickinson
 Number of poems in collection: 445

The only ghost I ever saw
 Was dressed in mechlin, -- so;
 He wore no sandal on his foot,
 And stepped like flakes of snow.
 His gait was soundless, like the bird,
 But rapid, like the roe;
 His fashions quaint, mosaic,
 Or, haply, mistletoe.

His conversation seldom,
 His laughter like the breeze
 That dies away in dimples
 Among the pensive trees.
 Our interview was transient,--
 Of me, himself was shy;
 And God forbid I look behind
 Since that appalling day!

In [31]: `grader.check("q2.7")`

Out [31]: **q2.7** passed! 100

Part 3: Math and Statistics

Linear algebra

Q3.1 — Shapes and conformability

Let A be a 4×7 matrix and B a 7×5 matrix, and let $C = AB$.

Give each answer as a tuple, e.g. `(1, 2)` :

- `C_dim` — the dimensions of C
- `AT_dim`, `AAT_dim`, `ATA_dim` — the dimensions of $A^\top$, $AA^\top$ and $A^\top A$
- `Ainv_dim` — the dimensions of M^{-1} , where M is a 6×6 invertible matrix

Finally, `non_conformable` — a list of the letters of those products below that **cannot be computed**:

- **A.** AA
- **B.** $A^\top A^\top$
- **C.** $A(A^\top A)$
- **D.** $(A^\top A)A^\top$

In [32]: `C_dim = (4, 5)`
`AT_dim = (7, 4)`
`AAT_dim = (4, 4)`
`ATA_dim = (7, 7)`

```
Ainv_dim = (6, 6)
non_conformable = ["A", "B"]
```

```
In [33]: grader.check("q3.1")
```

```
Out [33]: q3.1 passed! 🚀
```

In data science we constantly meet problems that can be written (or approximated) as a [system of linear equations](#).

Suppose you've started recording data about the books on your shelf:

Title	New/Cover Price (\$)	Years Since Publication	Has Underlining	Used Price (\$)
Perceptrons (Expanded)	35	35	1	15
Social Empiricism	20	21	0	11
Philosophical Investigations	42	13	1	27

Has Underlining is 1 if the book is marked up and 0 otherwise.

Assume — very naïvely — that a used price is a [linear combination](#) of those three features:

$$\beta_1 \cdot \text{new price} + \beta_2 \cdot \text{years since publication} + \beta_3 \cdot \text{has underlining} = \text{used price}$$

Collect the features into a matrix X , the used prices into a column vector y , and the unknown coefficients into a column vector β , and all three equations become one:

$$X\beta = y \quad \text{where} \quad X = \begin{bmatrix} 35 & 35 & 1 \\ 20 & 21 & 0 \\ 42 & 13 & 1 \end{bmatrix}, \quad y = \begin{bmatrix} 15 \\ 11 \\ 27 \end{bmatrix}$$

Because X here is square and full [rank](#), it has an [inverse](#), and multiplying both sides on the left by X^{-1} isolates β :

$$\begin{aligned} X\beta &= y \\ X^{-1}X\beta &= X^{-1}y \\ I\beta &= X^{-1}y \\ \beta &= X^{-1}y \end{aligned}$$

Q3.2 — Solve the system and make a prediction

First, use [NumPy's](#) array constructor, `np.array()`, to build X and y as shown above, then solve for `beta`.

Hints:

- y is a *column* vector: its `.shape` should be `(3, 1)`, not `(3,)`.
`.reshape()` is one way there.
- `np.linalg` has what you need to invert a matrix. The `@` operator does matrix multiplication.

Then predict the used price of a book that isn't in the table:

Title	New/Cover Price	Years Since Publication	Has Underlining	Used Price
Linear Algebra Done Right	44	7	0	?

Store its three features in `new_book` and the predicted price in `pred_price`.

```
In [34]: X = np.array([[35, 35, 1], [20, 21, 0], [42, 13, 1]])
y = np.array([[15], [11], [27]])

beta = np.linalg.inv(X) @ y

beta
```

```
Out[34]: array([[ 0.84156729],
               [-0.27768313],
               [-4.73594549]])
```

```
In [35]: new_book = np.array([[44, 7, 0]])
pred_price = new_book @ beta

print(f"Predicted price: ${float(pred_price.item()):.2f}")
```

Predicted price: \$35.09

```
In [36]: grader.check("q3.2")
```

```
Out[36]: q3.2 passed! 🍀
```

We will rarely be lucky enough to have a square data matrix, and the assumption that used prices are *exactly* linear combinations of these features, with no variation at all, is far too strong. The course relaxes both.

Multivariate calculus

Q3.3 — Partial derivatives for optimization

Calculus is how we optimize. Imagine a *loss* function $f(x, y)$ whose arguments are parameters we control, and we want the pair that makes it smallest:

$$\arg \min_{x, y} f(x, y)$$

A differentiable function's derivative is zero at an extremum. So take the partial derivative with respect to each parameter, set both to zero, and solve the resulting

system for candidate extrema. To confirm a candidate is a minimum, check that the second derivatives there are positive.

Use that strategy on

$$f(x, y) = xy + (y + 1)^2 + 2x^2 + 4$$

Show your work using $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ in the markdown cell below — one of the real conveniences of notebooks is that they render mathematics — then store the minimizing values in `x_min` and `y_min`. If your $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ is rusty, [Overleaf's guide to mathematical expressions](#) is a good short reference.

Partial derivatives. With $f(x, y) = xy + (y + 1)^2 + 2x^2 + 4$,

$$\frac{\partial f}{\partial x} = y + 4x, \quad \frac{\partial f}{\partial y} = x + 2(y + 1) = x + 2y + 2.$$

Critical point. Setting both to zero gives a linear system:

$$\begin{cases} 4x + y = 0 \\ x + 2y + 2 = 0 \end{cases}$$

The first gives $y = -4x$. Substituting into the second,

$$x + 2(-4x) + 2 = 0 \implies -7x + 2 = 0 \implies x = \frac{2}{7}, \quad y = -4 \cdot \frac{2}{7} = -\frac{8}{7}.$$

Second-derivative test. The second partials are constant:

$$f_{xx} = 4, \quad f_{yy} = 2, \quad f_{xy} = 1.$$

For a function of two variables it is not enough that f_{xx} and f_{yy} are positive; the mixed partial enters through the Hessian determinant:

$$D = f_{xx}f_{yy} - f_{xy}^2 = 4 \cdot 2 - 1^2 = 7 > 0, \quad f_{xx} = 4 > 0,$$

so the critical point is a local minimum. Because the Hessian is constant and positive definite, f is a convex quadratic and this local minimum is also the global one:

$$(x_{\min}, y_{\min}) = \left(\frac{2}{7}, -\frac{8}{7}\right), \quad f\left(\frac{2}{7}, -\frac{8}{7}\right) = \frac{27}{7}.$$

```
In [37]: x_min = 2 / 7
         y_min = -8 / 7
```

```
In [38]: grader.check("q3.3")
```

```
Out [38]: q3.3 passed! 🌟
```

Probability and statistics

Later in the course we model the value of interest y for each observation x as a **random variable** drawn from some family of **probability distributions** — Gaussian, binomial, and so on — whose parameters depend on x . Learning that dependence is the whole game. First, a refresher on probabilities themselves.

Q3.4 — Joint and conditional probability

(a) What is the probability of rolling three 3's in a row with a fair six-sided die? Store it in `dice_prob`.

(b) In your pocket are two coins: a **fair** one, and a **biased** one that lands heads twice as often as tails — that is, $P(H | B) = 2/3$. You take one at random, each equally likely, and flip it four times, observing

$$H, T, H, T$$

What was the probability of that sequence of outcomes? Store it in `coin_prob`.

(c) Now suppose you know you drew the biased coin. What is $P(H, H, H | B)$? Store it in `p_hhh_given_biased`, and show your work with *L^AT_EX* in the markdown cell below.

(b) The coin is drawn first, so condition on which one it is and use the law of total probability. Given a coin, the four flips are independent, so a sequence's probability is the product of its per-flip probabilities.

$$P(HTHT | F) = \left(\frac{1}{2}\right)^4 = \frac{1}{16}, \quad P(HTHT | B) = \left(\frac{2}{3}\right)^2 \left(\frac{1}{3}\right)^2 = \frac{4}{81}$$

$$\begin{aligned} P(HTHT) &= P(F) P(HTHT | F) + P(B) P(HTHT | B) \\ &= \frac{1}{2} \cdot \frac{1}{16} + \frac{1}{2} \cdot \frac{4}{81} = \frac{1}{32} + \frac{2}{81} = \frac{145}{2592} \approx 0.0559 \end{aligned}$$

(c) Now the coin is given, so there is nothing to average over. Conditioned on B the flips are independent and identically distributed with $P(H | B) = \frac{2}{3}$, so the joint probability factorises:

$$P(H, H, H | B) = P(H | B)^3 = \left(\frac{2}{3}\right)^3 = \frac{8}{27} \approx 0.2963$$

Note this is larger than the same event under the fair coin, $\left(\frac{1}{2}\right)^3 = \frac{1}{8}$, as it should be for a coin weighted towards heads.

```
In [39]: dice_prob = (1 / 6) ** 3

coin_prob = 0.5 * (1 / 2) ** 4 + 0.5 * (2 / 3) ** 2 * (1 / 3) ** 2

p_hhh_given_biased = (2 / 3) ** 3

print(f"(a) {dice_prob:.6f}")
```

```
print(f"(b) {coin_prob:.6f}")
print(f"(c) {p_hhh_given_biased:.6f}")
```

(a) 0.004630
 (b) 0.055941
 (c) 0.296296

In [40]: grader.check("q3.4")

Out [40]: q3.4 passed! 🌈

Q3.5 — Sampling from a distribution

NumPy can draw from probability distributions through its [random generator](#), built with `np.random.default_rng()`. Seeding it makes a run reproducible.

Let X be normally distributed with mean 4 and **standard deviation** 2 (not variance 2):

$$X \sim \mathcal{N}(\mu = 4, \sigma = 2)$$

(a) Draw `N` realizations of X into `sample`, then compute `sample_mean` and `sample_std`. Start with `N = 10` and increase it until both are within 0.1 of the true values.

(b) Compute $P(X > 7)$ two ways: `p_analytic`, from the normal CDF via `scipy.stats` (imported as `stats` at the top of the notebook), and `p_empirical`, as the fraction of 10^6 fresh draws that exceed 7. They should agree to about three decimal places — not exactly, because one is a simulation.

Hints:

- Check the docs for the generator's `normal()` method — note it takes the standard deviation, not the variance.
- `stats.norm(loc=..., scale=...).cdf(...)` gives $P(X \leq x)$.

In [41]: *# A fresh generator, so this question does not depend on draws taken above*
`rng = np.random.default_rng(seed=109)`

In [42]: `N = 1000`
`sample = rng.normal(loc=4, scale=2, size=N)`

`sample_mean = sample.mean()`
`sample_std = sample.std()`

`print(f"N = {N}")`
`print(f"sample mean: {sample_mean:.3f}")`
`print(f"sample standard deviation: {sample_std:.3f}")`

N = 1000
 sample mean: 4.016
 sample standard deviation: 1.963

```
In [43]: p_analytic = 1 - stats.norm(loc=4, scale=2).cdf(7)

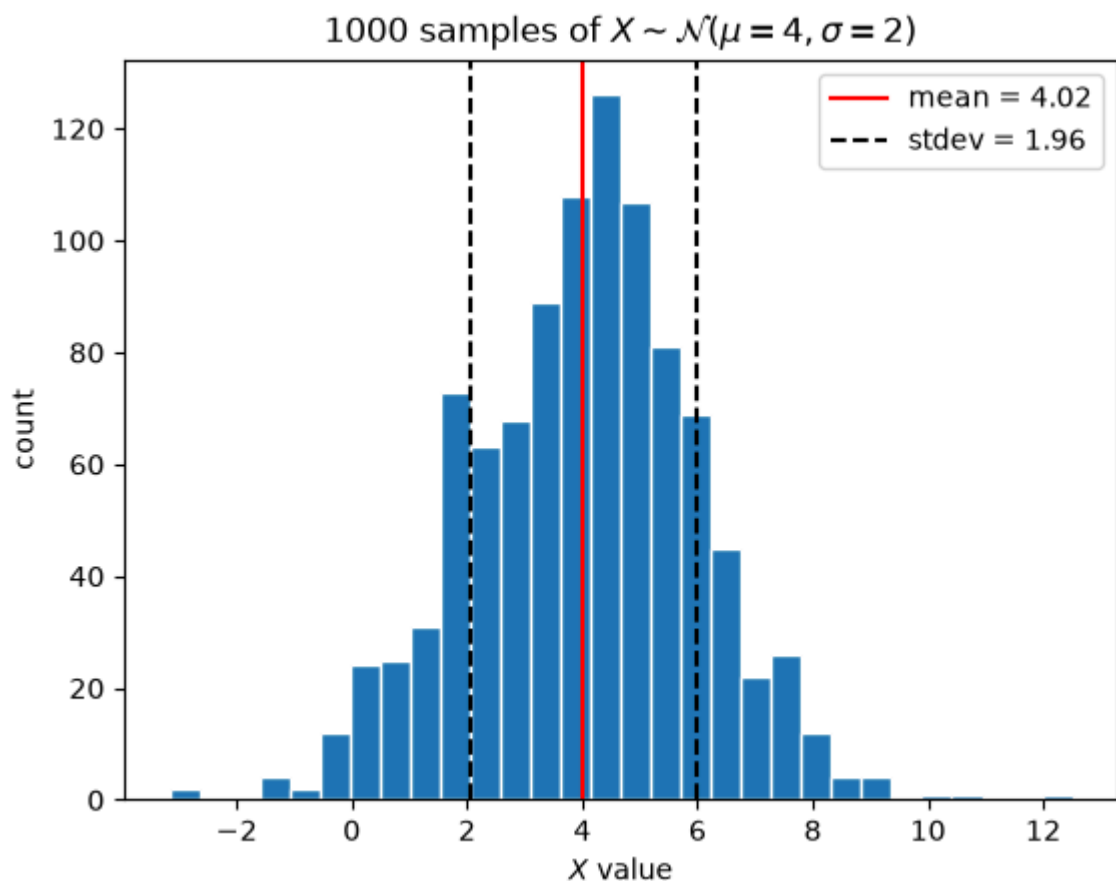
big_sample = rng.normal(loc=4, scale=2, size=10**6)
p_empirical = (big_sample > 7).mean()

print(f"analytic: {p_analytic:.6f}")
print(f"empirical: {p_empirical:.6f}")
```

analytic: 0.066807

empirical: 0.067198

```
In [44]: # provided plotting code
plt.hist(sample, bins=30, edgecolor="white")
plt.axvline(sample_mean, c="r", label=f"mean = {sample_mean:.2f}")
plt.axvline(sample_mean + sample_std, c="k", ls="--", label=f"stdev = {sa
plt.axvline(sample_mean - sample_std, c="k", ls="--")
plt.title(rf"{N} samples of $X \sim \mathcal{N}(\mu=4, \sigma=2)$")
plt.ylabel("count")
plt.xlabel(r"$X$ value")
plt.legend();
```



```
In [45]: grader.check("q3.5")
```

Out [45]: **q3.5** passed! 🙌

Q4 — Reflection

Required, and never scored. This is how we find out whether the assignment did its job, and how we know who to reach out to in week 1.

Fill in the three variables below, then answer the questions in the markdown cell.

```
In [46]: # Roughly how many hours did HW0 take you, start to finish?
hours_to_complete = 2

# How prepared do you feel for this course, on a scale of 1 (not at all)
preparedness = 4

# Where did you do most of this work?
# One of: "local", "fasondemand", "colab", "other"
where = "local"
```

1. Which part was hardest, and why?

- the multivariable took me most tough work. does not remember those details from years ago

2. Was there anything you could not do at all?

- no

3. What would you most like to review before the first week of class?

- python/pd/np basics

4. Optional: did you hit any trouble building your environment in Part 1? What happened?

- no

Answered Above

```
In [47]: grader.check("q4")
```

Out [47]: **q4** passed! 100

 **This concludes HW0. Thank you!**

Submitting

Do this first, every time: **Kernel → Restart Kernel and Run All Cells**, **then save**. It is what puts your output in the file, and it is checked. A notebook that was never run, whose cells were run out of order, or that **stops partway on a cell that raises** scores zero on the notebook assignment no matter what is written in it — **Run All** halts at the first error, so one broken cell blanks out everything below it.

That is a requirement to *run*, not to be correct: wrong answers earn full credit, and if a cell raises and you cannot fix it, put a plausible value in it with a comment saying what you were attempting. Part 2 is a chain, so an error usually points back at an earlier answer rather than the cell that reported it.

Then skim the notebook top to bottom; anything that dumps pages of raw text is worth narrowing, because a TF reads your PDF.

Then upload both files. They are two separate Gradescope assignments, due at the same time, and **you are not finished until both are in** — a completed assignment uploaded only as a notebook scores 70 out of 100.

1. **Homework 0 — Notebook** 📄 — this notebook, `cs1090a_hw0.ipynb`, worth **70**. An autograder replies within a couple of minutes.
2. **Homework 0 — PDF** 📄 — a PDF of it, worth **30**, read by a TF. After uploading, Gradescope asks you to **mark which pages each of six sections is on**:

| Mark this | With these pages | | :----- | :----- | | Part 1 — Setting the Stage | everything from the start of the PDF to the end of Part 1 | | Part 2 — Python Poetry Challenge | the whole of Part 2 | | Part 3 — Math and Statistics | the whole of Part 3 | | Q3.3 — Partial derivatives | just the page(s) with your derivation | | Q3.4c — Conditional probability | just the page(s) with your derivation | | Q4 — Reflection | just the page(s) with your reflection |

The three parts are marked whole because **a TF reads your entire submission** — that is where the *solid / shaky* verdicts on the PDF side come from. The last three are marked narrowly because those are the ones carrying the 30 points, and the page you mark is the page that gets graded. Q3.3 and Q3.4c fall inside Part 3, so their pages get marked twice. That is expected.

Gradescope's [guide to submitting a PDF assignment](#) walks through the page-selection screen. This is the same workflow you will use on HW1–HW5.

`SETUP.md` has the export instructions. Resubmit either file as often as you like before the deadline — the diagnostic you get back is the part worth reading. Just make sure both of them are there when the deadline passes.